Practical on time complexity.

① Find the time complexity for the following code

```
# include <stdio.h>
int factorial (int x)
{ if (x == 0 || n == 1)
  { return 1; }
  return x * factorial (x-1);
}
int main()
{ int x;
  scanf ("%d", &n);
  int result = factorial (x);
  printf ("%d", result);
  return 0;
}
```

If (n == 0 || x == 1)

{ return 1 ; }

O(1)

each recursive call multiplication by the recurse of factorial (x-1)

Time complexity.

$T(x) = T(x-1) + O(1)$

$T(x) = O(x)$

Code

```
#include <stdio.h>
int main()
{ int arr[] = {10,20,30,40,50}
  int n = size of (arr) / size of (arr[0
  int max = arr[0];
   for (int i=1 ; i<n; i++)
   { if(arr[i]>max) {
       max = arr[i];
   } }
Printf ("max element in the array = %d\n
        max);
   return 0;
}
```

The time complexity :.(O(x))

The loop iterates through the array one compare.

3 find the time complexity for the follow code.

```
#include <stdio.h>
int n, sum=0;
scanf ("%d", &n);
```

```
printf("sum of first %d national n
        %d \n:", n, sum);
    return 0;
}.
```

The value of n is read using scanf.
takes O(1) time.

Time complexity O(x)
The loop iterates times.

4.
```
for(int i=1; i<=n; i++){
    for(int j=1; j<=n; j++){
        sum = sum+i*j;
}
}
printf("sum = %d", sum);
```

Time complexity : O(n²)
Two nested loops, each iterating

5.
```
Sum=0;
for(int i=1; ;i<=n; ; i*=2){
    sum++;
}
printf("sum = %d", sum);
```
Time complexity : O(logn)

The loop variable double in each iteration.

int constant time complexity example.

```
int constant time example (int arr[],
    int size
    {   return arr[0];
    }
```

Time complexity : O(1)

```
int binary search (int arr[], int size;
                   int target)
{   int low=0;
    int high = size-1
    while (low <= high) {
    int mid = (low+high);
    if (arr[mid] == target) {
    return mid;
    } else if (arr[mid] < target){
        low = mid+1; }
        else {
            high = mid-1; }
        }
        return-1;
```

Time complexity : O(logn)
search space is halved in each iteration

8. int linear time example (int arr[], int size) {
int sum = 0;
for (int i=0; i<size; i++) {
sum += arr[i];
}
return sum;
}

Time complexity : O(n)
The loop iterates through array once.

9. void fun(int n) {
if (n < 5) {
printf ("let start");
}
else {
for (int i=0; i<n; i++) {
printf ("%d", i);
}
}
}

Time complexity : O(1) if n ≥ 5 ; other w
O(1) The loop runs n times only wh
n ≥ 5.

10 void fun (int n){
for (int i=1; i<=n; i++){
for (int j=1; j<=n; j++){
printf ("HLH");
}}}

Time complexity : O(logn)

The inner loop runs times for ea